AI Software Code Reviewer

& Secure Development Agent

Complete Architecture & Design Rationale

LangGraph | Semgrep | RAG (FAISS / TF-IDF) | Redis | Ollama / Gemini / Claude / GPT | Streamlit | Docker

9 specialist agents · one LangGraph pipeline · grounded, verified, human-approved fixes

Team Alpha • Architecture Deck • 21 slides

The problem & our solution

Why an automated, agentic reviewer

The problem

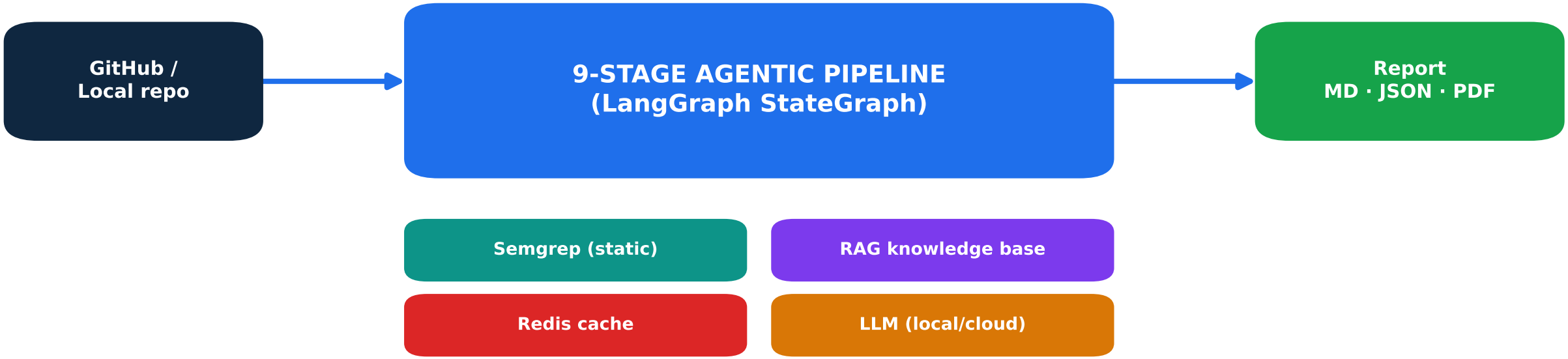
- Manual review is slow, inconsistent, and misses security bugs.
- Static tools flag issues but don't explain or fix them.
- Raw LLM fixes can hallucinate unsafe 'best practices'.
- High-risk changes must never be auto-applied without a human.

Our solution — one pipeline that:

Input: a GitHub repo (or local path) → Output: explainable report + verified fixes + PDF

High-level architecture

One shared state flows through 9 agents



Pluggable side services the pipeline calls into

UNDERSTAND	Ingestion · Static Analysis · Code-LLM Review
DETECT & GROUND	Merge & Dedupe · Secure-Coding RAG
FIX & CHECK	Explainable Fix Generation · Self-Verifier
APPROVE & REPORT	Human-in-the-Loop · Final Report

Why a multi-agent pipeline?

Design principle #1: separation of concerns

WHY WE CHOSE IT

- Each concern (scan, reason, fix, verify) is a hard problem on its own.
- One giant prompt would be unreliable and impossible to debug.
- Security demands checks & balances — a generator AND an independent verifier.

ADVANTAGES

- Each agent is small, testable, swappable.
- Failures are isolated & observable (per-node tracing).
- New capabilities = add a node, not a rewrite.
- Mirrors how human review teams work.

9 specialist agents, each doing one job well, coordinated by a shared state.

Why LangGraph for orchestration?

Design principle #2: an explicit state machine

WHY WE CHOSE IT

- We need a deterministic, ordered flow of agents — not free-form chatter.
- State must accumulate across stages (findings → fixes → approvals).
- We want to stream progress and trace every step for observability.

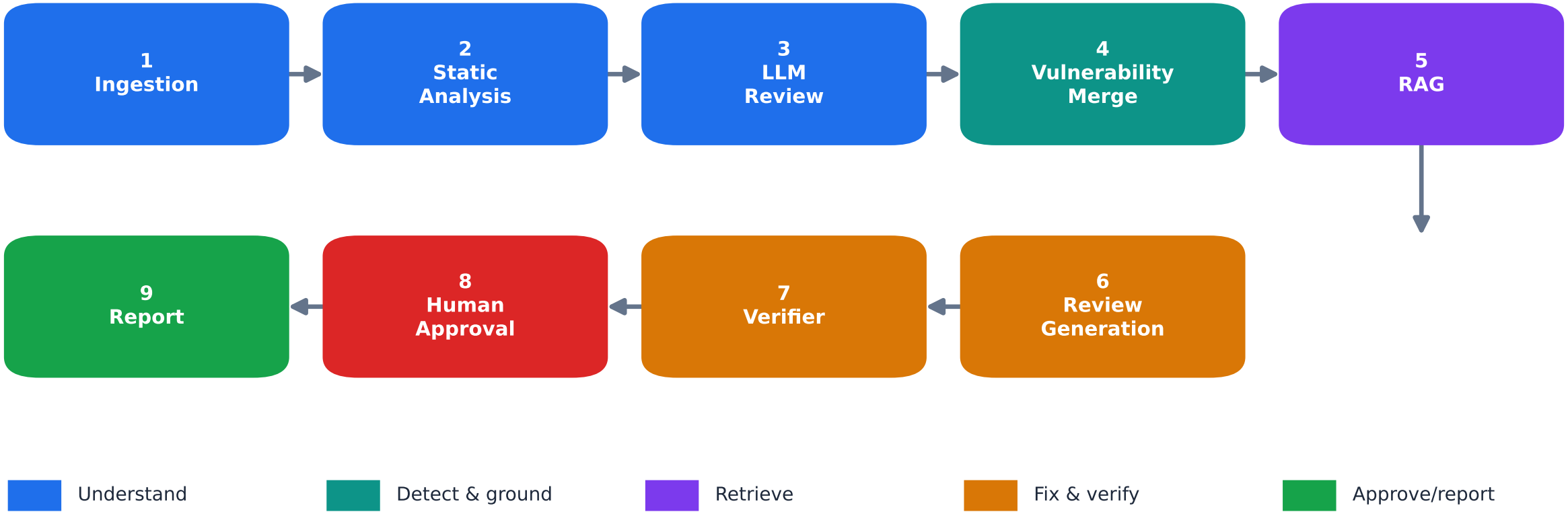
ADVANTAGES

- StateGraph = explicit nodes + edges; easy to read and reason about.
- Typed shared ReviewState passed node to node — no hidden globals.
- Native streaming drives the live UI progress bar, stage by stage.
- Each node wrapped with tracing spans.

graph.py wires 9 nodes as a linear StateGraph: ingestion → … → report.

The 9-node pipeline flow

Data moves through nine stages, in order



Linear & traceable: each node reads the previous node's output from the shared state.

The shared ReviewState

What each node adds to the state (agents/state.py)

One typed dict grows as it flows — every stage is a pure state → state function.

The 9 LangGraph nodes at a glance

Each node = one specialist agent (agents/*.py)

1	**Ingestion**	Walks the repo, reads files, detects language; honours batch/file caps.
2	**Static Analysis**	Runs Semgrep security rulesets + builds an AST code graph.
3	**LLM Review**	A Code-LLM reads each file for logic/semantic bugs rules miss.
4	**Vulnerability**	Merges & de-dupes Semgrep + LLM findings; splits security vs quality.
5	**RAG**	Retrieves grounded secure-coding guidance for each finding.
6	**Review Generation**	Writes a plain-language comment + a minimal, grounded fix.
7	**Verifier**	Syntax check, no-op check, Semgrep regression re-scan → confidence.
8	**Approval**	Routes CRITICAL/ERROR or unverified fixes to a human, with reasons.
9	**Report**	Assembles Markdown + JSON (+ PDF); computes the quality score.

Node 1 & 2 — Understand the code

Ingestion + Static Analysis

NODE 1 · INGESTION

- Discovers source files across 9 languages
- Reads contents, detects language by extension
- Applies the file cap / batch subset from the UI

→ files, file_contents, languages

NODE 2 · STATIC ANALYSIS

- Runs Semgrep (OWASP/CWE/secrets rulesets)
- Offline fallback ruleset if registry unreachable
- Builds an AST code graph (funcs, calls, sinks)

→ semgrep_findings, code_graph_summary

Why two lenses here?

Deterministic pattern matching (Semgrep) and structural context (code graph) give the LLM in Node 3 solid ground to reason over — not raw text alone.

Node 3 — Code-LLM semantic review

The intelligent lens (llm_review_agent.py)

- Summarizes each file's purpose
- Finds logic bugs, missing auth, race conditions, insecure design
- Catches what pattern-based tools structurally cannot
- Per-file calls run in parallel (ThreadPoolExecutor)

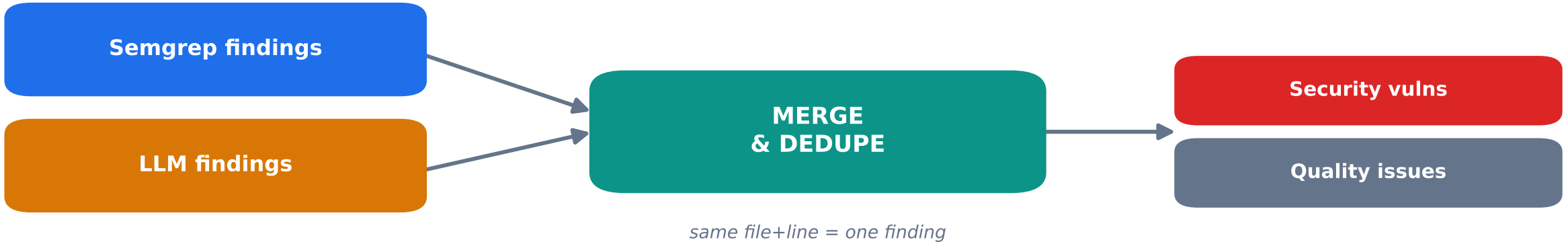
Pluggable LLM backend (utils/llm_client.py) — swap without code changes:

Ollama (local)	Google Gemini	Anthropic	OpenAI
`qwen2.5-coder:1.5b · llama3.2:1b`	`gemini-2.5-flash`	`claude-sonnet-4-6`	`gpt-4o-mini`

Local = free & private; Cloud = far faster and fully parallel. Chosen at runtime in the UI.

Node 4 — Vulnerability merge & dedupe

Combine the two lenses (vulnerability_agent.py)



WHY WE CHOSE IT

- Two detectors overlap — the same bug shouldn't be reported twice.
- Security and maintainability deserve different handling downstream.

ADVANTAGES

- One clean, de-duplicated finding list.
- category_breakdown drives the charts and the quality score.
- Severity tags feed the approval policy.

Node 5 & Why RAG

Retrieval-Augmented Generation for grounded fixes

RAG = retrieve real secure-coding guidance, then make the LLM fix WITH it.

WHY WE CHOSE IT

- Ungrounded LLM fixes hallucinate unsafe 'best practices'.
- Security guidance (OWASP/CWE) is authoritative and specific.
- We want fixes traceable to a source, not the model's guesswork.

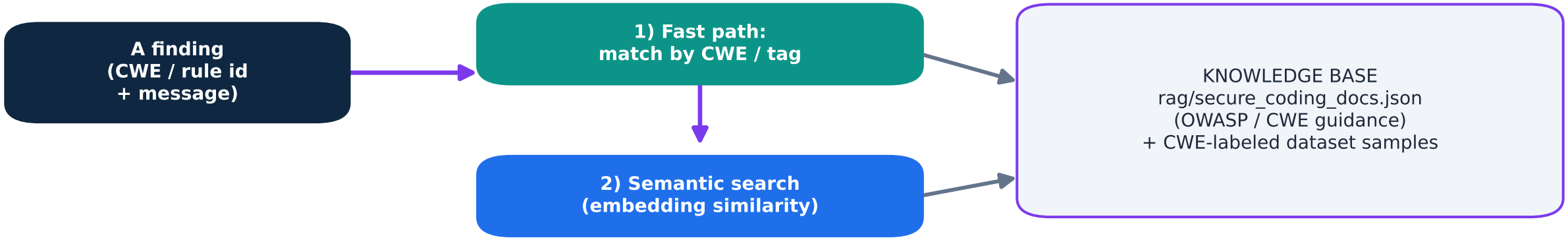
ADVANTAGES

- Fixes cite real CWE/OWASP remediation.
- Far fewer hallucinated / unsafe fixes.
- Explainable & auditable — every fix links to its guidance.
- OPTIONAL: a UI toggle turns it off to compare grounded vs. ungrounded.

RAG is optional & toggleable (USE_RAG). ON → fixes grounded in the KB; OFF → general best practice.

Inside the RAG engine

Knowledge base + two interchangeable backends



Two interchangeable backends (auto-selected):

Optional: sentence-transformers + FAISS

Default: scikit-learn TF-IDF (torch-free)

- Ships torch-free by default → small, fast Docker image; TF-IDF needs no model download.
- Add sentence-transformers + FAISS for semantic embeddings when higher recall is needed.
- Graceful fallback: if embeddings can't load, RAG still works offline via TF-IDF.

What's in the RAG knowledge base

The 12 grounded documents (rag/secure_coding_docs.json)

Each finding is matched to one of these curated OWASP/CWE remediation docs; the snippet is injected into the fix prompt.

CWE-89	SQL Injection	**CWE-78**	OS Command Injection
CWE-79	Cross-Site Scripting (XSS)	**CWE-20**	Improper Input Validation
CWE-502	Untrusted Deserialization	**CWE-327**	Weak / Broken Crypto
CWE-798	Hard-coded Credentials	**CWE-259**	Hard-coded Password
CWE-732	Incorrect Permissions	**OWASP A09**	Insufficient Logging & Monitoring
Quality	Secure & Maintainable Error Handling	**Quality**	Avoid Magic Numbers & Duplication

10 security (CWE/OWASP) remediation docs + 2 code-quality guides. Extend by adding entries to the JSON.

Node 6 — Explainable fix generation

review_generation_agent.py

For every merged finding, the Code-LLM produces:

- A plain-language explanation — WHY this is a problem (for a mid-level dev).
- A minimal, targeted code fix — no unrelated rewrites.
- Grounded in the RAG guidance retrieved for that finding.
- Per-finding calls run in parallel; prompts use repo-relative paths so the cache reuses across runs.

→ suggested_fixes (original code, fixed code, explanation) + review_comments

Covers the 'explainable code review + automated fix' objective — with Explainable-AI reasoning.

Node 7 — Verifier / self-reflection

The AI checks its own work (verifier_agent.py)

Three independent checks on every generated fix:

Syntax check	Does the fixed code parse (AST for Python)?
No-op check	Did it actually change the code, or fake a fix?
Regression re-scan	Re-run Semgrep — is the original rule now gone?

WHY WE CHOSE IT

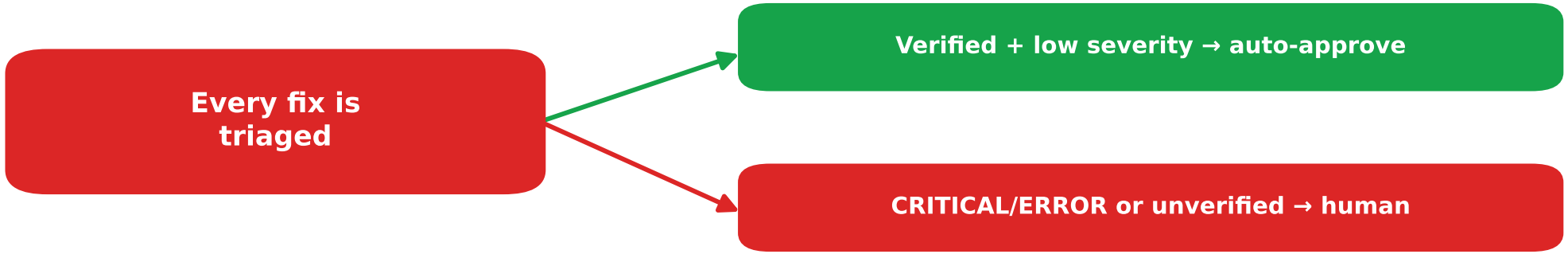
- LLM fixes can be hallucinated, incomplete, or no-ops.
- Security fixes must be proven, not trusted.

ADVANTAGES

- A confidence score per fix & per run.
- Unverified fixes get flagged → sent to a human, never auto-applied.
- Resilient: one bad fix can't crash it.

Node 8 — Human-in-the-loop approval

Never auto-apply high risk (approval_agent.py)



Each decision carries a plain-language reason, shown in the UI.

WHY WE CHOSE IT

- High-risk code changes must have a human gate (compliance & trust).
- Reviewers need to see WHY each fix was auto-approved or escalated.

ADVANTAGES

- Nothing high-risk is applied silently.
- Approve / Reject / override buttons in the Streamlit dashboard.
- 'Require approval for ALL' toggle for strict mode.

Node 9 — Final report

Assemble every artifact (report_agent.py)

REVIEWSTATE → REPORT

- Bug & vulnerability summary
- Code quality score (0–100)
- Confidence score
- Markdown + JSON export
- Security (CWE/OWASP) report
- Suggested + verified fixes
- Human-approval log
- Downloadable PDF (reportlab)

Quality score = smooth exponential decay on issue density

so a vulnerable repo lands in a low-but-informative band instead of a flat 0 — a usable signal.

Why Redis?

Caching LLM responses for speed & cost

LLM calls are the slow, expensive part. Redis caches every completion.

WHY WE CHOSE IT

- Local CPU inference is slow; cloud calls cost money & latency.
- Re-reviewing the same file+model should be free and instant.
- A demo/hackathon box needs zero-setup infra.

ADVANTAGES

- Key = (provider, model, system, prompt, max_tokens) → exact-match reuse.
- Repeat runs of a repo are near-instant.
- redislite embedded fallback → no server to install; degrades to no-op safely.
- 7-day TTL; live hit/miss stats in the UI.

utils/redis_cache.py: standalone Redis → embedded redislite → transparent no-op (never a hard dep).

Datasets used

Grounding & benchmarking the vulnerability knowledge

Folded into the RAG corpus (CWE-labeled vulnerable/fixed samples):

Devign	C/C++ function-level vuln detection (CodeXGLUE)
Big-Vul (MSR'20)	CVE-labeled real-world vulnerabilities
Juliet (NIST)	Synthetic CWE weakness test cases
DiverseVul	Large-scale diverse vulnerable functions

Additional reference benchmarks (README):

- **CodeXGLUE** — code intelligence
- **CodeSearchNet** — semantic search
- **OWASP Benchmark** — tool scoring
- **ManySStuBs4J** — Java bug fixes
- **SAP Project-KB** — Java vuln benchmark
- **CVEfixes** — CVE fix commits

rag/dataset_loader.py streams samples into the KB; fetch_datasets.py can cache real HF datasets offline.

Technology stack — and why each

Every technology in the system

LangGraph	Explicit, streamable agent state machine
Gemini / Claude / GPT	Fast, high-quality cloud LLM options
scikit-learn TF-IDF	Torch-free default RAG retrieval
Redis / redislite	LLM-response cache (zero-setup fallback)
Plotly + matplotlib	Interactive charts + PDF chart images
Python ast	Code-graph structural context
Ollama	Local Code-LLMs (qwen2.5-coder, llama3.2)
Semgrep	Deterministic OWASP/CWE static analysis
sentence-transf. + FAISS	Optional semantic embedding search
Streamlit	Interactive review dashboard
reportlab	Finalized PDF report generation
Docker + compose	Reproducible, portable deployment

Streamlit UI & deployment

Where a reviewer actually works

The Streamlit dashboard (app.py):

- Point at a GitHub URL (auto-clone) or local path
- Live, stage-by-stage progress as agents run
- Findings highlighted by severity + clickable file:line links to GitHub
- Interactive charts: severity/type pies, gauge, treemap, per-file bars
- Human-approval tab with reasons + Approve/Reject/override
- One-click PDF report; batch mode for large repos; model & cache controls

Run locally

```
streamlit run src/app.py
```

Run in Docker

```
docker compose up --build
```

Find → Ground (RAG) → Fix → Verify → Human-approve → Report. Explainable, cached, and portable.